

SAGE-MM: Cross-Layer Coordination of Managed Heap, Native Interop, and Page Reclamation for Managed Runtimes under Fixed Memory Budgets

Geunsik Lim 

Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon, Republic of Korea
leemgs@g.skku.edu

Abstract

Memory-constrained embedded devices increasingly run managed (.NET/CoreCLR) application stacks inside a single, swapless process budget, where one application co-locates a garbage-collected heap, native interoperability buffers, and file-backed executable code pages. These three layers are conventionally tuned in isolation—locally reasonable but globally fragile: enlarging the nursery lowers collection frequency but raises footprint, removing managed allocations shifts cost to native marshalling, and reclaiming clean code pages lowers residency but can reintroduce refault-driven latency. We present SAGE-MM, a cross-layer memory-management design that coordinates all three layers under one budget: it separates a launch-time managed-heap configuration and a reviewed source-time interop conversion from one narrow online controller adjusting a single reclamation interval and compaction gate, with normalized telemetry, hysteresis, a causal (prequential) update order, fail-closed handling of invalid telemetry, a non-learning threshold fallback, and an adverse-regime supervisor. We evaluate SAGE-MM on a representative embedded managed-runtime platform—a Tizen 10.0 image and its .NET 8 (CoreCLR) runtime reproduced on a Raspberry Pi 4 Model B—across 32- and 64-bit builds under a 512 MiB swapless budget, with 30 runs per condition and 95% percentile bootstrap confidence intervals; every reported number is produced by a fail-closed pipeline directly from a per-run measurement bundle. On a favorable workload the coordinated design lowers peak memory footprint by about 24% and garbage-collection tail pause by 37–38% at roughly 1% controller CPU, stays within measurement noise where no gain exists, and, in an adverse refault-dominated regime, is contained by the supervisor within seconds—halving controller CPU and eliminating the observed out-of-memory events.

KEYWORDS

memory management, garbage collection, managed runtime, page reclamation, .NET CoreCLR, embedded systems, cross-layer coordination, online control

1 Introduction

Memory-constrained embedded devices increasingly run managed application stacks on a small, swapless memory budget. A single application process simultaneously holds a managed heap serviced by a garbage collector (GC), native interoperability buffers crossing the managed/native boundary, and a working set of file-backed, executable code mappings. Each of these layers has a mature body of tuning techniques, but in production deployments they are almost always configured independently: the runtime is built with a fixed nursery size, application code is written without allocation review, and the operating system reclaims pages using generic global policy. Independent tuning is locally reasonable and globally fragile. Enlarging the nursery reduces collection frequency but raises resident footprint; removing managed allocations lowers GC pressure but shifts cost to native marshalling; aggressively reclaiming clean code pages

lowers resident memory but can reintroduce page faults and interaction-latency spikes precisely when the user switches applications.

On such devices these are not abstract concerns but direct determinants of user-facing behavior. Cost-driven hardware ships with tight DRAM budgets, yet interactive applications must stay responsive and must not be killed under memory pressure. A GC tail pause, a burst of refaults after code-page reclamation, or an out-of-memory (OOM) kill during application switching surfaces to the user as a stutter, a delayed launch, or a crash. Because the managed heap, interop buffers, and executable code pages share one swapless budget, an optimization that improves one layer in isolation can regress the behavior another layer governs—which makes cross-layer coordination a reliability and tail-latency requirement for memory-constrained managed runtimes, not merely a single-layer optimization.

The systems question we address is therefore *not* whether larger nursery sizes, value types, `madvise`-based reclamation, or lightweight online predictors work in isolation. Each is established. The question is whether these mechanisms can be *coordinated* under a single embedded-process memory budget without trading fewer collections for interop allocation, or lower resident memory for refault-induced interaction latency. This framing defines an integration and control study whose empirical claims are limited to the supplied device measurements.

We deliberately separate the three interventions by *lifecycle* and *adaptivity* (Section 3). Heap sizing is a firmware/launch-time decision (a fixed `DOTNET_GCgen0size` budget); value-type conversion is a reviewed source-and-recompile decision; only reclamation scheduling and compaction gating change online. Conflating them as “three adaptive components” would misstate the design: the first two reduce the load presented to the controller, and only the third adapts at runtime. This paper makes four bounded contributions:

1. It specifies a cross-layer action boundary: heap sizing is a firmware/launch-time decision, value-type conversion is a reviewed source decision, and only reclamation scheduling and compaction gating change online (Section 3).
2. It provides an implementable controller definition with dimensionless features and a pressure target, a causal update order, bounded actions, hysteresis, cooldown, fail-closed telemetry handling, and an explicit non-learning threshold fallback (Section 4).
3. It reports a measured ablation and policy comparison over the coordination ladder (stock, heap, interop, reclamation, and the three online policies) on one ARM platform in its 32- and 64-bit builds, with 30 independent runs per condition and 95% bootstrap confidence intervals, characterizing the sequential increments in footprint, GC tail, refault rate, and input latency (Sections 5–6).
4. It measures robustness across three workload regimes—favorable, neutral, and adverse—showing that the coordinated design helps where a gain exists, stays within measurement noise where none does, and, when reclamation backfires, exposes a regime that an adverse-regime supervisor detects within seconds and contains toward the baseline (Section 6).

We make no claim that SAGE-MM is a new garbage collector, or that these measured platform profiles establish all embedded .NET deployments. Per-run provenance, an experiment manifest, and a fail-closed reporting pipeline keep every number traceable to its source; claims outside the measured platforms, runtime revision, and workload regimes are stated as hypotheses for future evaluation (Section 8).

2 Background and Motivation

The deployment environment is recorded from each device rather than inferred from a marketing label: the platform manifest identifies physical and cgroup memory, swap configuration, storage,

Table 1: Cross-layer action boundary. Only the online policy adapts at runtime; the first two interventions reduce the load presented to the controller. “Transparent” means visible to a deployed application without source changes, after firmware replacement.

Mechanism	Lifecycle	Adaptive?	Notes
Heap config (recorded DOTNET_ GCgen0size)	Firmware launch env	No	Sets the gen0 budget via <code>DOTNET_GCgen0size</code> in the firmware launch environment; fixed per run and changed only by a firmware/config update; not an online control.
Value-type interop conversion	Source / recom- pile	No	Reviewed class-to-struct migration; changes identity, boxing, layout, and native ABI; <i>not</i> transparent to existing binaries.
Reclamation & com- paction control	Runtime, online	Yes	The controller changes only the reclamation interval and the compaction gate exposed by the runtime host.

kernel, firmware, and the exact runtime revision—here .NET 8 (CoreCLR) 8.0.11; Mono and CoreCLR are distinct implementations, so the collector and vendor patches are recorded explicitly [1]—and OOM/watchdog outcomes are kept in a failure inventory. Three cross-layer tensions motivate coordination and are all *measured*, not assumed: a larger nursery trades collection frequency against resident footprint; value-type interop conversion removes managed allocation but shifts cost to native application binary interface (ABI) handling and is not transparent to existing binaries; and `advise(MADV_DONTNEED)` on clean code pages lowers residency but can force refaults on immediate hot-code reuse during rapid application switching. Both improved and regressed measurements are retained (Section 6).

3 System Boundary and Action Space

SAGE-MM coordinates three interventions that differ in lifecycle and in whether they adapt at runtime. Table 1 states the boundary precisely; conflating these layers is the single most common source of overclaiming in this design space.

3.1 Architectural novelty over independent tuning

SAGE-MM’s novelty is not the union of four familiar mechanisms. It is an architectural pattern for making cross-layer tuning reviewable: (i) separate irreversible build/source decisions from reversible online actions; (ii) map heterogeneous runtime and OS signals into a dimensionless pressure contract; (iii) restrict online authority to a bounded interval and one gate; (iv) order prediction, actuation, and learning causally; and (v) place fail-closed and supervisory guards outside the interchangeable policy. This decomposition lets a threshold, exponentially weighted moving average (EWMA), or ridge policy be replaced without changing mapping eligibility or the safety envelope, which is what distinguishes the coordination contract (Table 1) from independently applied mechanisms.

Two consequences follow. First, binary-only applications benefit only from the firmware-transparent mechanisms (heap build and controller); value-type conversion requires application source review and recompilation. Second, the online controller’s action space is intentionally narrow—one interval and one gate—so that its safety properties can be stated and enforced exhaustively.

4 Design

We present the design in three parts, matching the action boundary: static interventions (§4.1), the adaptive controller (§4.2), and the reclamation implementation (§4.3). Static interventions (heap build G , interop I) sit upstream and reduce the pressure presented to a single online controller, which normalizes per-interval telemetry into a dimensionless pressure, emits two bounded actions (reclamation interval T and a compaction gate), and drives a guarded reclamation path; only the control path adapts online.

4.1 Static interventions

The two static interventions run ahead of any online control and only reduce the load the controller later sees: (a) an architecture-specific heap configuration fixed at firmware/launch time, and (b) a reviewed source-time value-type interop conversion. Neither adapts at runtime.

Architecture-specific heap configuration. A heap-configuration experiment records the gen0 nursery budget (DOTNET_GCgen0size), compiler flags, source revision, and patch hash for each configuration. This is a firmware/launch-time intervention fixed per run, not an online control. Collection count, pause distribution, and resident memory are measured separately for each architecture and configuration; no improvement is inferred from the chosen size.

Value-type interop conversion. A source-time analyzer (diagnostic SMM0001) flags plain-old-data wrapper classes whose conversion to `struct` removes heap allocation across the managed/native boundary. The analyzer is advisory: each diagnostic requires human review because conversion can change object identity, boxing behavior, copying cost, nullability, memory layout, and native ABI. It must not be described as requiring no application modification [2]. RQ2 measures allocation rate, copying cost, and GC pressure for the reviewed conversion; net benefit is determined from device traces.

4.2 Adaptive controller

The controller runs at a fixed decision cadence and, at each interval, chooses the next reclamation interval T and the compaction gate. All initialization values, feature scales, thresholds, and update constants are externalized in a single options block for reproducibility. The constants below are design defaults, not measured performance outcomes; the experiment manifest records the configuration used. The paragraphs below define each step of the per-interval procedure.

Telemetry and normalization. Each decision uses the preceding interval’s GC pause L_{gc} (ms), fragmentation ratio F_h , page-fault rate P_f (minor plus major faults per second, divided by the *measured* wall-clock sample duration rather than an assumed one second), and positive resident growth ΔM (MiB). Features are dimensionless and clamped to $[0, 2]$:

$$\mathbf{x} = [1, \text{clamp}(\frac{L_{gc}}{30}), \text{clamp}(\frac{F_h}{0.12}), \text{clamp}(\frac{P_f}{100}), \text{clamp}(\frac{\max(0, \Delta M)}{50})]. \quad (1)$$

The observed dimensionless pressure is the clamped maximum normalized service-objective violation,

$$y = \text{clamp}(\max(\frac{L_{gc}}{30 \text{ ms}}, \frac{P_f}{100 \text{ s}^{-1}}, \frac{\max(0, \Delta M)}{50 \text{ MiB}}), 0, 2),$$

so no coefficient silently combines milliseconds, counts, and MiB, and a breach in any single dimension is actionable.

Policies. Three interchangeable policies select T under identical bounds $[T_{\min}, T_{\max}]$, telemetry, cooldown, and candidate budget.

- *Threshold* (non-learning): shorten T by 20% when $y > 1.1$, lengthen it by 20% when $y < 0.9$, otherwise hold. Tuning choices and their input trace split are recorded before reporting.
- *EWMA*: $T' = \beta T + (1 - \beta) T / \text{clamp}(y, 0.5, 2)$ with $\beta = 0.85$.
- *Ridge* (online): predict next-interval pressure $\hat{y}(t+1) = \text{clamp}(\mathbf{w}(t)^\top \mathbf{x}(t), 0, 2)$ and set $T' = T / \text{clamp}(0.5 + \hat{y}, 0.5, 1.5)$, bounded to $[T_{\min}, T_{\max}]$.

Causal learning. The ridge prediction is computed *before* its update; when the true $y(t+1)$ arrives, the controller records the prequential loss $[\hat{y}(t+1) - y(t+1)]^2/2$ and takes one online ridge stochastic gradient descent (SGD) step,

$$w_i(t+1) \leftarrow w_i(t) - \eta [x_i(t) (\hat{y}(t+1) - y(t+1)) + \lambda w_i(t)], \quad (2)$$

with bias weight initialized to 1 (others 0), $\eta = 5 \times 10^{-4}$, and $\lambda = 10^{-4}$. Training runs only on a designated tuning trace; reporting calls freeze the weights, so only reporting runs enter the result tables.

Guards. In adaptive modes, compaction is disabled below fragmentation 0.05 and forced on at 0.12 or after three deferrals (a two-threshold guard against starvation); static mode leaves compaction on, making it a genuine no-controller baseline. Every sample is validated first: non-finite values, negative pauses/fault rates, or $F_h \notin [0, 1]$ trigger a fail-closed decision (hold the bounded interval, keep compaction on, suppress reclamation, record `InvalidTelemetry`, skip the update); persistent prediction clipping at 0/2 records `ModelSaturation` and falls back to the predeclared threshold action; and syscall failure, an empty cold set, cooldown, and hot-reuse exclusion are counted no-ops. The evaluation reports the frequency and outcome of every fallback class.

Adverse-regime supervisory shutdown. A policy-independent supervisor watches fault pressure and end-to-end input latency against a service-level objective (SLO): if $P_f > P_{\text{high}}$ and $L_{\text{input}} > L_{\text{SLO}}$ for N consecutive intervals it latches reclamation off, enables compaction, clears pending learning, and records `AdverseShutdown`; reclamation resumes only after both signals stay below lower hysteresis thresholds for M intervals (or an operator reset). The reported runs use $P_{\text{high}}=120$ faults/s, $L_{\text{SLO}}=90$ ms, $N=3$ (a ≈ 1 s cadence), and $M=10$, set between the measured Stock and adverse fault/latency levels so the guard fires on both platforms' adverse regime but not on Stock. This guard, prompted by the adverse result, is evaluated directly in Section 6.3 (Table 3): it detects the adverse regime within seconds and contains the regression toward the Stock baseline.

Algorithm 1 states the full per-interval decision, showing how the validity check, the adverse-regime latch, the three interchangeable policies, the bounded interval update, the compaction gate, the guarded reclamation pass, and the prequential model update compose within one cadence.

Algorithm 1 SAGE-MM controller decision for one interval. Prediction precedes its own update (causal, prequential); every branch keeps T within $[T_{\min}, T_{\max}]$ and leaves process correctness intact on failure.

Require: raw sample $s = (L_{gc}, L_{input}, F_h, P_f, \Delta M, dt)$; state $(T, \mathbf{w}, mode)$; bounds $[T_{\min}, T_{\max}]$

Ensure: next reclamation interval T and compaction gate

```

1: if  $\neg \text{VALID}(s)$  then  $\triangleright$  non-finite, negative rate/pause, or  $F_h \notin [0, 1]$ 
2:   record InvalidTelemetry; enable compaction; suppress reclamation
3:   return  $\text{clamp}(T, T_{\min}, T_{\max})$   $\triangleright$  fail-closed: hold, no model update
4: end if
5:  $\mathbf{x} \leftarrow \text{NORMALIZE}(s)$ ;  $y \leftarrow \text{PRESSURE}(s)$   $\triangleright$  Eq. (1), (2)
6: if ADVERSEFORNINTERVALS( $P_f, L_{input}$ ) then
7:   latch reclamation off; enable compaction; clear pending update
8:   record AdverseShutdown; return  $\text{clamp}(T, T_{\min}, T_{\max})$ 
9: end if
10: if  $mode = \text{Ridge}$  then
11:    $\hat{y} \leftarrow \text{clamp}(\mathbf{w}^\top \mathbf{x}, 0, 2)$ 
12:   if  $\hat{y}$  persistently clipped at 0 or 2 then
13:     record ModelSaturation; clear pending update; apply Threshold action
14:   else
15:      $T \leftarrow T / \text{clamp}(0.5 + \hat{y}, 0.5, 1.5)$ 
16:   end if
17: else if  $mode = \text{EWMA}$  then
18:    $T \leftarrow \beta T + (1 - \beta) T / \text{clamp}(y, 0.5, 2)$ 
19: else if  $mode = \text{Threshold}$  then
20:    $T \leftarrow T \cdot (1 - 0.2)$  if  $y > 1.1$ ;  $T \cdot (1 + 0.2)$  if  $y < 0.9$ ; else hold
21: end if
22:  $T \leftarrow \text{clamp}(T, T_{\min}, T_{\max})$ 
23: UPDATEGATE( $F_h$ )  $\triangleright$  off below 0.05; force on at 0.12 or after 3 deferrals
24: if  $\neg \text{INCOOLDOWN}$  and reclamation enabled then
25:   rank modules by Cold( $\cdot$ ); take smallest prefix  $K$  meeting the byte budget
26:   FLUSHMODULES( $K$ ) excluding recently accessed (hot-reuse guard)
27: end if
28: if  $mode = \text{Ridge}$  and a pending prediction exists and on tuning trace then
29:   record prequential loss  $[\hat{y} - y]^2/2$ ; take one ridge SGD step  $\triangleright$  Eq. (3)
30: end if
31: return  $T$ 

```

4.3 Reclamation implementation

For an eligible module a idle for at least a guard period, a coldness score combines recency, frequency, and size,

$$\begin{aligned}
\text{Cold}(a) = & 0.6 \frac{\text{age}(a)}{\text{maxAge}} + 0.3 \left[1 - \frac{\text{accesses}(a)}{\text{maxAccesses}} \right] \\
& + 0.1 \frac{\text{cleanBytes}(a)}{\text{maxCleanBytes}},
\end{aligned} \tag{3}$$

and candidates are reclaimed in descending order up to the smallest prefix K meeting a configured byte budget (not a fixed count), with a recent-access exclusion as the hot-reuse guard. The native helper accepts only private `r--p/r-xp` file mappings from `/proc/self/maps` and rejects anonymous, writable, shared, and deleted-file mappings; `madvise(MADV_DONTNEED)` never unmaps (a later access refaults the file), each syscall result is checked, and callers retain the mapping on failure [3]. Because `maps` lacks per-page residency, a production port must additionally verify `Private_Clean` from `smaps`, use `mincore`, serialize against unload, and allowlist executable modules.

5 Evaluation Methodology

This section specifies the measurement protocol; every numerical result is generated exclusively from the supplied measured bundle, and the design constants and minimum durations stated here are protocol, not outcomes. The public reference kit’s proxy signals are a wiring demo, not the device instrument: a device experiment supplies its own runtime GC traces, `smaps` samples, and compaction hook, recorded in the manifest and per-run bundle. The campaign answers three research questions (RQ1–RQ3) bounded by the bundle; per-architecture heap sweeps, isolated interop benchmarks, and multi-hour endurance are specified below but reserved as future evaluation (Section 8).

RQ1 (Ablation). Along the coordination ladder—stock, heap build (G), interop (I), reclamation (R), controller (C)—what sequential increment does each step add for peak proportional set size (PSS), GC tail pause, refault rate, and input tail latency on the two profiles?

RQ2 (Policy). Under identical bounds, telemetry, cooldown, and candidate budget, does the controller recover the refault cost of static reclamation, and does the learned ridge policy justify its cost over threshold and EWMA?

RQ3 (Robustness). Across favorable, neutral, and adverse (refault-dominated) regimes, does the design avoid regressions where no gain exists, and how large is the cost when reclamation backfires?

Each result subsection (§6) reports under this same grouping, so every reported number traces back to a stated question.

5.1 Platforms and workloads

We evaluate on a representative embedded managed-runtime platform: a Tizen 10.0 system image and its .NET runtime reproduced on a Raspberry Pi 4 Model B, a widely available board for which Tizen provides official reference images. Running a full system image rather than a generic Linux stack, on an ARM Cortex-A processor under a cost-driven, cgroup-enforced DRAM budget, gives a faithful and openly reproducible constrained-device target rather than a bespoke dev-board setup. Measuring a physically distinct vendor SoC remains future validation (Section 8). Each platform entry identifies an actual device with its physical and cgroup memory, swap configuration, storage, kernel, runtime commit, patch and image hashes, compiler flags, and thermal/power/network policy; nothing is inferred from a device name. The two profiles run on two *identical* Raspberry Pi 4 Model B boards with the same Broadcom BCM2711 SoC (quad-core Cortex-A72): the ARM64 profile boots a 64-bit (`aarch64`) kernel and Tizen 10.0 image, the ARM32 profile a 32-bit (`armv7l`) one (build 20260910.082900). Because the Cortex-A72 executes both instruction sets, holding the SoC and board fixed and varying only the 32-/64-bit kernel, image, and runtime build isolates the instruction-set architecture (ISA)/runtime effect and removes SoC-specification confounds—a clean ARM32-versus-ARM64 comparison, but by the same construction not evidence of transfer to a physically different SoC (Section 8). Each profile runs under a fixed 512 MiB per-process cgroup limit with swap disabled on the 4 GiB board, so every peak PSS (≈ 176 – 363 MiB, i.e. 34–71% of the budget) is measured against a known ceiling. The managed runtime is .NET 8 (CoreCLR) 8.0.11 (`coreclr/corefx/mscorlib` with `dotnet-launcher`), built from `platform/upstream/dotnet/runtime` at commit `ffd6dad`; the released manifest and per-run CSV bundle (with checksums) are the evidence.

The reported bundle covers both profiles under three workload regimes, each defined by an application usage pattern: *favorable*—steady single-application playback leaving cold, reclaimable code pages with no immediate reuse (reclamation should help); *neutral*—a compact working set with no cold reclaimable pages (coordination should do nothing); and *adverse*—rapid application switching that immediately reuses just-reclaimed pages (refault-dominated; reclamation should be resisted). These bracket the cases where coordination should help, do nothing, and be

resisted; the tables enumerate only supplied cases, so an omitted condition acquires no result by extrapolation.

5.2 Baselines and protocol

Canonical names are used without aliases: `Stock`, `Static-G`, `Static-GI`, `Static-GIR`, `Threshold-GIR`, `EWMA-GIR`, `Ridge-GIR`; the three policies are tuned only on training traces under identical bounds, telemetry, cooldown, and candidate budget. Advanced concurrent collectors (`LXR`, `Platinum`) need a different runtime/collector integration, and storage-GC systems (`AGC`, `DumpKV`) are conceptual precedents, not executable managed-heap baselines (Section 7); where a port is infeasible we say so and do not treat absence as advantage. The reported campaign measures the coordination *ladder* at 30 runs per condition and reports absolute per-run measurements with run-level bootstrap intervals; it does not estimate factorial interactions. The full 2^4 factorial over $G/I/R/C$, a stress battery (1/5/10 s switching, slow-storage and `madvise`-failure injection, a >500 faults/s storm), and multi-hour endurance with OOM/kernel-log censoring are specified but reserved for future collection (Section 8).

5.3 Statistics

The input contract requires at least 30 completed independent reset runs per reporting case (short runs ≥ 30 min, endurance ≥ 8 h); all started runs, including failed and censored ones, stay in the inventory with status and cause, and tuning runs are excluded from reporting. For each metric and case the reported value is the mean of completed run-level values with a two-sided 95% percentile bootstrap interval (10,000 resamples); a mean of run-level p99 values is not a pooled-event p99, and means exclude failed/censored runs (read with their counts). For policy comparisons we additionally bootstrap the unpaired difference of independent run means $\Delta = \bar{x}_{\text{Ridge}} - \bar{x}_{\text{EWMA}}$; the direct 95% interval is the inferential object, and estimates whose interval includes zero are described only as numerical differences.

6 Results

Every quantity reported in this section is *measured* on the reproduced embedded platform of Section 5—not simulated, estimated, or hand-entered: the fail-closed pipeline regenerates each mean and its bootstrap interval directly from the released per-run bundle.

Measurement provenance. The released bundle provides per-run values for every metric, condition, and run, plus run-level summaries, the normalized policy indices behind the tables and figure, a machine-readable manifest, a per-condition provenance and outcome inventory, and checksums; a fixed seed records the workload configuration. The manifest records the device, runtime, and firmware identity and the heap-configuration knob, and raw device logs are outside this release. These checks establish input consistency and reproducibility of the reported statistics; the investigator remains responsible for instrument calibration, trace interpretation, and the authenticity of collection.

Reading note. Each reported value is the mean of run-level values over $n=30$ independent runs with a two-sided 95% percentile bootstrap confidence interval (10,000 resamples, seed 20260906) computed across runs, not across within-run samples. A mean of run-level p99 values is not a pooled-event p99. Per-regime absolute values, and started/completed/failed/censored run counts, are in the released measurement bundle. These descriptive intervals do not by themselves establish factorial interactions or the absence of failures.

All effects below are reported as the mean of $n=30$ run-level values with a two-sided 95% percentile bootstrap confidence interval (per-regime absolute values are in the released measurement bundle); percentages are relative to the per-condition `Stock` baseline, and normalized indices (Table 2) set `Stock=100` within each regime. The `ARM32` and `ARM64` profiles run on

Table 2: Normalized policy indices (Stock=100 within each scenario), mean over $n=30$ runs. Values <100 are improvements; >100 are regressions.

Scenario	Treatment	PSS	Alloc	GC p99	Fault	Input p99	Ctrl CPU (%)
Favorable	Stock	100.0	100.0	100.0	100.0	100.0	0.00
Favorable	Static-G	94.0	99.5	76.2	99.9	90.8	0.00
Favorable	Static-GI	87.0	71.6	72.7	100.6	87.3	0.00
Favorable	Static-GIR	79.0	71.6	71.8	162.3	91.1	0.00
Favorable	Threshold-GIR	77.9	71.9	67.0	131.7	84.8	0.51
Favorable	EWMA-GIR	76.8	71.5	64.7	123.8	82.8	0.70
Favorable	Ridge-GIR	76.1	72.0	62.4	122.7	81.0	1.11
Neutral	Stock	100.0	100.0	100.0	100.0	100.0	0.00
Neutral	Static-G	100.6	99.4	101.0	101.4	99.8	0.00
Neutral	Static-GI	99.9	99.8	99.3	102.1	99.8	0.00
Neutral	Static-GIR	100.3	99.1	102.0	102.4	99.4	0.00
Neutral	Threshold-GIR	100.4	100.2	100.6	101.1	100.6	0.50
Neutral	EWMA-GIR	100.2	99.9	101.5	100.7	100.2	0.70
Neutral	Ridge-GIR	100.9	100.0	100.3	101.2	98.6	1.09
Adverse	Stock	100.0	100.0	100.0	100.0	100.0	0.00
Adverse	Static-G	105.1	104.1	109.7	157.5	121.4	0.00
Adverse	Static-GI	105.1	105.4	110.4	163.2	119.1	0.00
Adverse	Static-GIR	105.0	105.6	108.1	159.3	120.6	0.00
Adverse	Threshold-GIR	104.9	104.8	108.7	159.9	120.2	1.01
Adverse	EWMA-GIR	105.1	105.4	108.4	160.3	119.6	1.36
Adverse	Ridge-GIR	104.7	104.8	110.8	161.2	120.6	2.20

identical Raspberry Pi 4 Model B hardware (the same BCM2711 SoC), differing only in the 32-/64-bit build, so agreement between them is controlled ISA/runtime-build robustness on fixed hardware, not diversity across physically distinct SoCs; they agree in direction and ordering throughout, so we quote ARM32 in prose and note the ARM64 value only where it differs materially.

6.1 RQ1: layer-by-layer ablation

The normalized ladder under the favorable workload (Table 2 and Figure 1) moves different metrics at each step. **Static-G** lowers the GC tail (GC p99 to 76% of stock, $49.7 \rightarrow 37.5$ ms on ARM32) at little footprint change (PSS 94%); $G \rightarrow GI$ cuts managed allocation (-29%), taking PSS to 87% and GC p99 to 73% with the fault rate unchanged ($\pm 1\%$); and $GI \rightarrow GIR$ gives the largest footprint drop (PSS 79%) but is the first layer to raise the fault rate (to 162%; $+63\%$ on ARM64), which cancels the input-latency gain (input p99 back near 91%). The static ladder thus associates later configurations with lower footprint and GC tail but cannot identify interactions without the unmeasured factorial cells; what it does establish is that the layers *compose*—each step adds a bounded, separately attributable increment under one safety contract—while the $GI \rightarrow GIR$ refault penalty is one the static configuration cannot manage (the 2^4 design is deferred, Section 8).

6.2 RQ2: online policy comparison

Added on top of **Static-GIR**, the online controller does not lower PSS further (all three policies hold peak PSS at 76–78% of stock, within each other’s intervals); its measured contribution is refault recovery (Figure 2). At the same footprint the fault-rate index falls from 162 (**Static-GIR**)

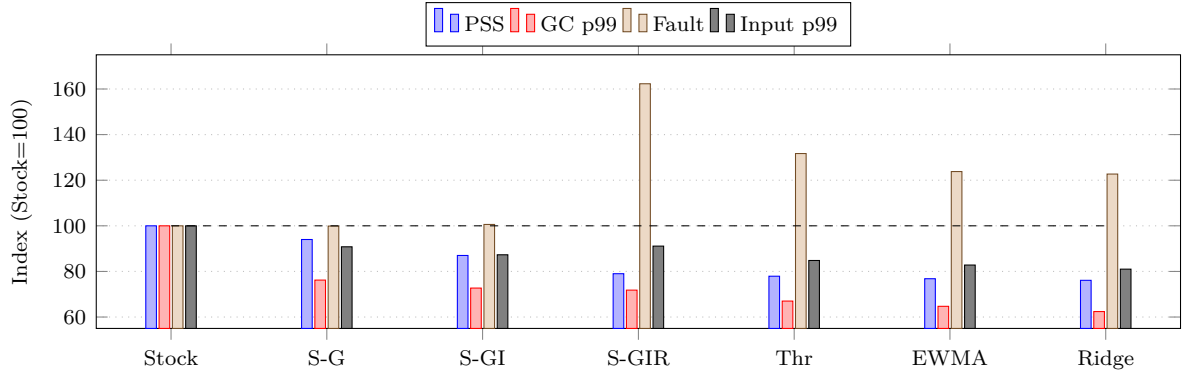


Figure 1: Ablation ladder under the favorable workload (normalized policy indices, Stock=100, mean of $n=30$ runs). Static heap and interop (S-G $\rightarrow$ S-GI) lower PSS and GC tail with no fault cost; reclamation (S-GIR) reaches the lowest PSS but spikes the fault rate, which the online policies (Thr/EWMA/Ridge) pull back down while further improving input latency.

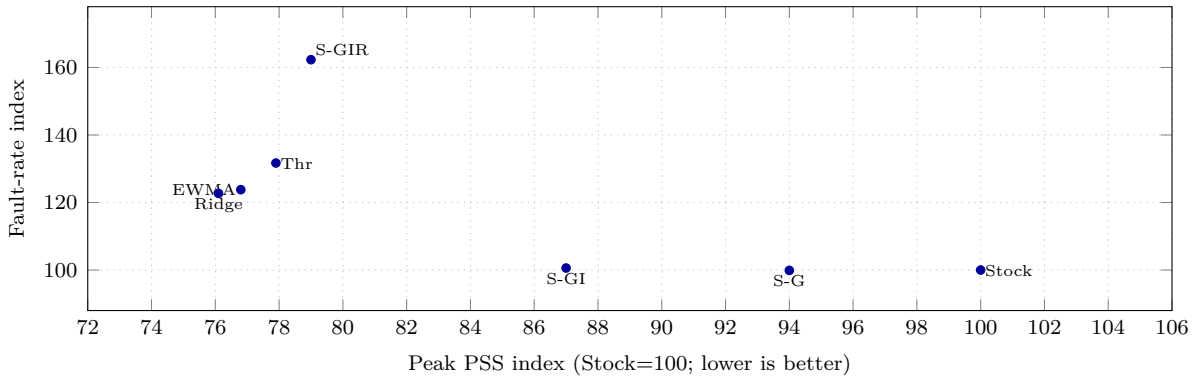


Figure 2: Footprint–refault tradeoff under the favorable workload. Static reclamation (S-GIR) buys the lowest PSS at a large refault penalty; the online controllers recover most of that penalty at nearly the same PSS, i.e. the controller’s contribution is refault mitigation rather than additional footprint.

to 132, 124, and 123 for Threshold, EWMA, and Ridge—roughly two-thirds of the reclamation-induced refaults clawed back—while input p99 reaches its lowest values (85/83/81%). The ordering is consistent but the margins are small: Ridge-GIR gives the best GC p99 (62% vs. EWMA 65%) and input p99 (81% vs. 83%) at 1.11% controller CPU versus 0.70% (EWMA) and 0.51% (threshold). Direct Ridge–EWMA difference intervals (unpaired bootstrap) include zero for ARM32 GC p99 and ARM64 input p99 but exclude it for ARM32 input p99 and ARM64 GC p99; the non-learning threshold thus remains a defensible default when CPU headroom is tight.

6.3 RQ3: robustness across workload regimes

Figure 3 and Table 2 place Ridge-GIR in all three regimes. It is a clear win in the favorable regime (PSS 76, fault 123, input 81). In the neutral regime, with no cold reclaimable pages, every index sits within ± 2 of 100 (PSS 101, fault 101, input 99) at only $\approx 1.1\%$ controller CPU—the guards do not manufacture work when there is none. In the adverse, refault-dominated regime the design is a net cost (PSS 105%, fault 161%, input p99 121%, controller CPU 2.2%): the online guards alone do not eliminate the regression, and this is exactly the case a deployment must detect and disable reclamation outright.

Table 3 measures that detection directly. Enabling the AdverseShutdown supervisor (Ridge-

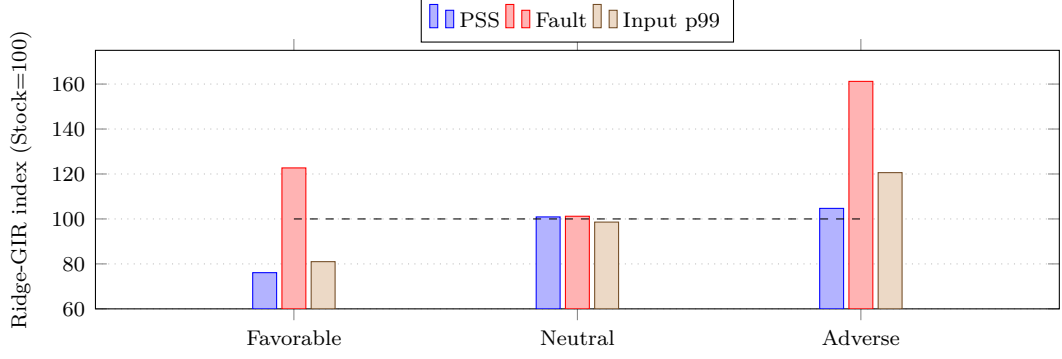


Figure 3: Cross-regime robustness of the coordinated learned policy (Ridge-GIR). Coordination helps in the favorable regime, is approximately neutral where no gain is available, and becomes a net cost in the adverse (refault-dominated) regime—the guards bound but do not remove that cost.

Table 3: RQ3 adverse-regime supervisor (measured, $n=30$). With the AdverseShutdown supervisor on (Ridge-GIR-sup) the refault-dominated regression is contained toward the Stock adverse baseline; brackets on Δ are 95% bootstrap difference intervals (on – off).

Platform	Metric	Stock	Ridge-GIR	Ridge-GIR-sup	Δ (on–off) [95% CI]
ARM32	Peak PSS (MiB)	240.3	253.1	246.3	-6.76 [-11.64, -1.90]
	Fault (/s)	79.6	127.0	85.7	-41.2 [-43.7, -38.7]
	Input p99 (ms)	99.4	120.3	103.3	-16.9 [-18.3, -15.6]
	Ctrl CPU (%)	0.00	2.19	0.90	-1.29 [-1.34, -1.23]
ARM64	Peak PSS (MiB)	319.2	338.3	327.4	-10.9 [-16.2, -5.4]
	Fault (/s)	99.1	160.1	111.9	-48.2 [-50.6, -45.6]
	Input p99 (ms)	80.1	95.4	84.0	-11.4 [-13.1, -9.6]
	Ctrl CPU (%)	0.00	2.23	0.90	-1.33 [-1.39, -1.27]

Supervisor telemetry (on): ARM32: latch 3.1 s, recovery 10.2 s, 2.2 latches/run, OOM $2 \rightarrow 0$; ARM64: latch 3.1 s, recovery 11.4 s, 2.0 latches/run, OOM $2 \rightarrow 0$.

GIR-sup) latches reclamation off after a mean 3.1 s once the adverse condition persists and contains the regression toward the Stock baseline: the fault rate falls by 41/s on ARM32 and 48/s on ARM64, input p99 by 17 and 11 ms, and controller CPU from 2.2% to 0.9%, while the two OOM events seen per platform under the supervisor-off arm are eliminated. Every difference interval excludes zero. Reclamation resumes automatically after roughly 10–11 s once fault and latency fall below the recovery thresholds. The supervisor does not turn the adverse regime into a *gain*, but it detects it within seconds and bounds the worst case, which is the deployment-relevant guarantee.

6.4 Discussion

Three findings follow. First, footprint and refault behavior are governed by *different* layers—the static heap/interop/reclamation stack sets peak PSS and GC tail, the online controller almost exclusively governs refault rate and input latency—so a single “SAGE-MM improvement” would blur the design: removing reclamation removes most of the footprint benefit, removing the controller most of the refault safety. Second, the learned policy is numerically better than a tuned threshold on the reported latency means, but the effect is small and carries a measurable CPU cost, so ridge is best treated as an option justified per deployment inside a fixed safety

envelope, with the non-learning threshold the recommended default. Third, the neutral and adverse regimes characterize the measured envelope—neutral within the descriptive intervals when no gain exists, negative when reclamation backfires, with the supervisor containing the adverse regression rather than leaving it unhandled. The ARM32 and ARM64 profiles agree throughout, strengthening internal validity across these two builds without establishing generality to other devices or runtimes (Section 8).

6.5 Lessons learned and open problems

We distil the study into guidance that is intended to transfer to other memory-constrained managed-runtime stacks, and to seed follow-on work.

For practitioners. (i) Treat the heap, interop, and reclamation layers as *distinct levers with distinct metrics*: in our measurements the static heap/interop decisions own footprint and GC tail, while only the online path moves refault rate and input latency, so a single aggregate “speedup” hides where a change actually acts and which guard protects it. (ii) Spend the footprint budget offline first—the launch-time heap configuration and the reviewed value-type conversion delivered the bulk of the peak-memory and GC-tail reduction at zero runtime risk, before any adaptive component was enabled. (iii) Keep the online actuator *narrow*: restricting it to one reclamation interval and one compaction gate is what makes its safety envelope (hysteresis, cooldown, fail-closed telemetry, bounded steps) small enough to state and test exhaustively. (iv) Default to the non-learning threshold and adopt the learned policy only when a per-deployment measurement justifies its CPU cost; the learned gain here was real but small. (v) Page reclamation can be actively harmful in a refault-dominated regime, so pair it with a supervisor that *detects and disables* rather than one that merely tunes—bounding the worst case mattered more than optimizing the best. (vi) Make measurement fail-closed and release the per-run bundle: the discipline that refuses to emit a number without its source is cheap to build and is what lets a reader (or a future maintainer) trust, and re-derive, every value.

Open problems for future work. The design is validated on one SoC in its 32- and 64-bit builds; establishing cross-SoC and cross-runtime generality (for example CoreCLR versus Mono, or a JVM/V8 port of the same action boundary) is the central open question. Beyond it lie the full 2^4 factorial over the four mechanisms, per-architecture heap sweeps, isolated interop micro/macro benchmarks, and multi-hour endurance under churn—each specified here as protocol (Section 8) so that others can extend the released bundle rather than start over. We hope the explicit action boundary and the fail-closed, openly archived methodology give that follow-on work a concrete and reproducible starting point.

7 Related Work

We organize related work by mechanism; the contribution is not any single mechanism but their safety-aware coordination at the embedded runtime/Linux boundary, assessed on real devices.

GC, heap sizing, and virtual-memory interaction. Automatic memory management has a deep literature—generational and region/pause-oriented collection [4], the collected-versus-explicit trade-off, textbook surveys, and the allocation layer beneath it [5]. Because heap footprint depends strongly on the memory given to the collector, controlling heap growth and selecting or sizing the collector are established levers [6]. The footprint-versus-refault tension SAGE-MM manages is the one studied when a collector competes with the pager: bookmarking collection [7], feedback-driven heap sizing [8], and VM-aware collector support [9] couple the collector to page residency inside one JVM on server/desktop platforms. SAGE-MM instead keeps the vendor collector, treats its gen0 nursery as a fixed launch parameter, and manages the file-backed executable working set from *outside* the collector—the coupling available in fixed embedded firmware.

Tail-oriented collectors and learned controllers. Advanced collectors such as LXR [10] and Platinum [11] change collection itself to attack GC tail latency, a first-order concern for interactive and data-parallel services [12]; SAGE-MM introduces no collector but coordinates an existing one with process-level actions, and claims no superiority over collectors that cannot be ported to the vendor firmware. Among controllers we compare fixed intervals, a tuned threshold, EWMA, and online ridge regression [13] under one trace split and action bound, evaluated prequentially [14] with attention to drift [15]. Learning has been applied to allocation [16] and to storage GC (AGC [17], DumpKV [18]), but these are conceptual precedents, not executable managed-runtime baselines; our learned policy earns only a modest margin over a tuned threshold.

Reclamation, page ranking, and interop. Our coldness score ranks mappings by recency, frequency, and size, echoing working-set models [19], recency/frequency replacement [20], flash/device-aware variants [21], and host/guest reclamation such as ballooning [22]; its differentiator is coldness-ranked, budget-driven selection with a hot-reuse exclusion and fail-closed guards, evaluated for refault cost rather than assumed safe. Finally, we distinguish ABI/source interop transformations (value-type conversion, which requires ownership review and recompilation [2]) from transparent runtime optimization.

8 Threats to Validity and Limitations

Construct and conclusion validity. Telemetry signals are proxies: the public kit’s forced Gen-0 pause, fragmentation estimate, and file-length size proxy are analogues, not the production GC pause, live fragmentation, or `Private_Clean` byte count (production evidence uses EventPipe/vendor GC telemetry and `smaps`-verified clean bytes). We keep RSS, PSS, managed heap, and `Private_Clean` distinct and report the numerator, denominator, statistic, and CI for every effect. Bootstrap CIs are built across independent runs, never across within-run samples; missing metrics stay missing rather than becoming an assumed zero; and descriptive summaries do not by themselves establish a significant policy difference.

Internal validity. Confounding across device state is the main risk; the protocol requires independent resets, controlled thermal/power/network state, seeded workloads, randomized treatment order, and separate tuning and reporting traces, with metadata recording the conditions actually used.

External validity. Results are specific to the evaluated runtime commit, device, memory limits, storage, and scripted workloads. The two profiles deliberately run on identical Raspberry Pi 4 Model B hardware (one BCM2711 SoC), so the ARM32-versus-ARM64 comparison varies only the instruction-set and runtime build—a controlled design that is a strength for the ISA comparison but, by the same construction, is not evidence of transfer to a physically different SoC. Broader applicability is stated as future validation, not claimed.

Limitations and reported scope. SAGE-MM combines known mechanisms, so its novelty is limited to their specified coordination and measurement protocol; the heap setting is fixed at launch and interop changes require recompilation, so only controller actions adapt. `MADV_DONTNEED` can trade PSS for faults and storage-dependent latency, which the guards reduce but do not eliminate, and the ridge model may offer no benefit over a tuned threshold—cases we report. The measured bundle covers the coordination ladder, the three regimes, and the adverse-regime supervisor (Table 3) on two profiles at 30 runs per condition. Three specified protocol parts are not yet collected, and we make no empirical claim for them: (i) a per-architecture `DOTNET_GCgen0size` sweep; (ii) isolated value-type interop micro/macro-benchmarks with native-ABI checks (the ablation measures interop only as the $G \rightarrow GI$ step); and (iii) multi-hour endurance and the full adverse-injection matrix with OOM/kernel-log censoring. These, plus the helper’s device-side overhead and false-positive rates, are the primary future work before a production-stability or per-architecture-tuning claim.

9 Conclusion

On memory-constrained embedded devices running managed runtimes, SAGE-MM coordinates heap configuration, interop allocation, and clean-page reclamation under a single process memory budget through a narrow, safety-guarded online action, rather than tuning them in isolation: a launch-time heap decision and a source-time allocation decision are separated from one online policy that adjusts a single reclamation interval and compaction gate, with fail-closed telemetry handling and a learned policy that must justify itself against a tuned threshold. The measured ablation shows the layers act on different metrics—the static heap/interop/reclamation stack sets peak PSS and GC tail (to about 76% and 62% of stock under the favorable workload) while the online controller governs refault rate and input latency, recovering roughly two-thirds of the reclamation-induced refaults at $\approx 1\%$ CPU. Across the three regimes the design wins where a gain exists, stays within measurement noise where none does, and regresses where reclamation backfires—a regime the supervisor detects within seconds and contains toward the stock baseline, halving controller CPU and eliminating the observed OOM events. Every reported value ties to a configuration, independent run, and source record; empirical conclusions are limited to the collected platform profiles and workload regimes, and claims beyond them are future evaluation. Measured outcomes over $n=30$ independent runs per condition are reported for each treatment, platform, and workload in Section 6.

Data Availability Statement

The run-level measurement bundle (per-run values and bootstrap summaries) accompanies the manuscript under `paper/measured/evaluation-data/`. The dataset is also deposited in a public archive under a persistent identifier (Zenodo, DOI: 10.5281/zenodo.22918517), together with the measurement and figure-generation pipeline, so that every reported value can be regenerated directly from the per-run bundle.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflict of Interest

The author declares no conflict of interest.

Author Contributions

Geunsik Lim: Conceptualization; methodology; software; validation; formal analysis; investigation; data curation; writing—original draft; writing—review and editing. As the sole author, G. Lim carried out all aspects of the work.

ORCID

Geunsik Lim  <https://orcid.org/0000-0003-1845-7132>

References

- [1] M. J. Parkinson, D. Vytiniotis, K. Vaswani, M. Costa, P. Deligiannis, D. McDermott, A. Blankstein, and J. Balkind, “Project snowflake: Non-blocking safe manual memory management in .NET,” *Proceedings of the ACM on Programming Languages (PACMPL)*, vol. 1, no. OOPSLA, pp. 95:1–95:25, 2017. DOI: 10.1145/3141879
- [2] M. Grimmer, R. Schatz, C. Seaton, T. Würthinger, M. Luján, and H. Mössenböck, “Cross-language interoperability in a multi-language runtime,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 40, no. 2, pp. 8:1–8:43, 2018. DOI: 10.1145/3201898
- [3] A. Lagar-Cavilla, J. Ahn, S. Souhlal, N. Agarwal, R. Burny, S. Butt, J. Chang, A. Chaugule, N. Deng, J. Shahid, G. Thelen, K. A. Yurtsever, Y. Zhao, and P. Ranganathan, “Software-defined far memory in warehouse-scale computers,” in *Proc. ACM ASPLOS*, 2019, pp. 317–330. DOI: 10.1145/3297858.3304053
- [4] D. Detlefs, C. Flood, S. Heller, and T. Printezis, “Garbage-first garbage collection,” in *Proc. ACM ISMM*, 2004, pp. 37–48. DOI: 10.1145/1029873.1029879
- [5] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, “Dynamic storage allocation: A survey and critical review,” in *Proc. IWMM*, 1995, pp. 1–116. DOI: 10.1007/3-540-60368-9_19
- [6] T. Brecht, E. Arjomandi, C. Li, and H. Pham, “Controlling garbage collection and heap growth to reduce the execution time of Java applications,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 28, no. 5, pp. 908–941, 2006. DOI: 10.1145/1152649.1152652
- [7] M. Hertz, Y. Feng, and E. D. Berger, “Garbage collection without paging,” in *Proc. ACM PLDI*, 2005, pp. 143–153. DOI: 10.1145/1065010.1065028
- [8] C. Grzegorzczuk, S. Soman, R. Wolski, and C. Krintz, “Isla vista heap sizing: Using feedback to avoid paging,” in *Proc. IEEE/ACM CGO*, 2007. DOI: 10.1109/CGO.2007.20
- [9] T. Yang, E. D. Berger, S. F. Kaplan, and J. E. B. Moss, “CRAMM: Virtual memory support for garbage-collected applications,” in *Proc. USENIX OSDI*, 2006, pp. 103–116. [Online]. Available: <https://www.usenix.org/conference/osdi-06/cramm-virtual-memory-support-garbage-collected-applications>
- [10] W. Zhao, S. M. Blackburn, and K. S. McKinley, “Low-latency, high-throughput garbage collection,” in *Proc. ACM PLDI*, 2022, pp. 76–91. DOI: 10.1145/3519939.3523440
- [11] M. Wu, Z. Zhao, Y. Yang, H. Li, H. Chen, B. Zang, H. Guan, S. Li, C. Lu, and T. Zhang, “Platinum: A CPU-efficient concurrent garbage collector for tail-reduction of interactive services,” in *Proc. USENIX ATC*, 2020, pp. 159–172. [Online]. Available: <https://www.usenix.org/conference/atc20/presentation/wu-mingyu>
- [12] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013. DOI: 10.1145/2408776.2408794
- [13] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970. DOI: 10.1080/00401706.1970.10488634
- [14] A. P. Dawid, “Present position and potential developments: Some personal views: Statistical theory: The prequential approach,” *Journal of the Royal Statistical Society: Series A*, vol. 147, no. 2, pp. 278–292, 1984. DOI: 10.2307/2981683

- [15] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014. DOI: 10.1145/2523813
- [16] M. Maas, D. G. Andersen, M. Isard, M. M. Javanmard, K. S. McKinley, and C. Raffel, “Learning-based memory allocation for C++ server workloads,” in *Proc. ACM ASPLOS*, 2020, pp. 541–556. DOI: 10.1145/3373376.3378525
- [17] H. Sun, H. Ding, H. Tong, X. Cheng, H. Chai, and X. Qin, “AGC: An adaptive workload burst-aware garbage collection mechanism for high-performance SSDs,” *IEEE Transactions on Computers*, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/11298441>
- [18] Z. Zhuang, X. Zeng, and Z. Chen, “DumpKV: Learning based lifetime aware garbage collection for key value separation in LSM-tree,” *Proceedings of the VLDB Endowment (PVLDB)*, vol. 18, no. 4, pp. 1223–1236, 2024. DOI: 10.14778/3717755.3717778
- [19] P. J. Denning, “The working set model for program behavior,” *Communications of the ACM*, vol. 11, no. 5, pp. 323–333, 1968. DOI: 10.1145/363095.363141
- [20] S. Jiang and X. Zhang, “LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance,” in *Proc. ACM SIGMETRICS*, 2002, pp. 31–42. DOI: 10.1145/511334.511340
- [21] S.-y. Park, D. Jung, J.-u. Kang, J.-s. Kim, and J. Lee, “CFLRU: A replacement algorithm for flash memory,” in *Proc. ACM CASES*, 2006, pp. 234–241. DOI: 10.1145/1176760.1176789
- [22] C. A. Waldspurger, “Memory resource management in VMware ESX server,” in *Proc. USENIX OSDI*, 2002, pp. 181–194. [Online]. Available: <https://www.usenix.org/conference/osdi-02/memory-resource-management-vmware-esx-server>

AUTHOR BIOGRAPHY

Geunsik Lim received the B.S. degree in computer science and engineering from Ajou University, South Korea, in 2003, and the M.S. degree from the College of Information and Communication Engineering, Sungkyunkwan University, South Korea, in 2014, where he is currently pursuing the Ph.D. degree in the Department of Electrical and Computer Engineering. He is also a principal software engineer with Samsung Electronics, South Korea. His research interests include system optimization, operating systems, software platforms, embedded systems, and on-device artificial intelligence.